

Python Fundamentals (Part 1)

Complete Learning Guide: Output, Input, Variables, Data Types & Operators

Author: Golam Maula Lincoln

GitHub Profile: <https://github.com/gmlincoln>

Visit GitHub Profile 

1. Output in Python

The first and easiest task in Python is printing (or outputting) information on the screen. We perform this using the built-in `print()` function.

Think of a function as a specialized helper that executes a specific job. Here, `print()` helps display text, numbers, calculation results, or formatted strings onto the console.

Syntax & Examples

Anything inside single quotes (`'...'`) or double quotes (`"..."`) is treated as a **string** and printed exactly as is.

```
# Printing string literals
print("hello world") # Output: hello world

# Printing numerical values and calculations
print(3.14)          # Output: 3.14
print(2 + 3)         # Output: 5

# Printing multiple values on the same line
print("hello world", "from PRIME") # Output: hello world from PRIME
```

Python Character Set

A character set is the complete collection of characters recognized by Python:

- **English Letters:** A-Z and a-z
- **Digits:** 0-9
- **Special Symbols:** +, -, *, /, %, _, etc.
- **Whitespaces & Escape Sequences:** Blank space, tab (`\t`), newline (`\n`).
- **ASCII & Unicode:** Supported as part of string literals and data.

2. Variables & Identifier Rules

A **variable** is a container (or labeled box) that stores data or values in memory. You can store data, modify it, or reference it later in your code.

```
name = "Shradha"
age = 30
PI = 3.14
word = "artificial intelligence"

print("value of PI =", PI) # Output: value of PI = 3.14
```

Variable Naming Rules (Identifiers)

1. Must start with a letter (a-z, A-Z) or an underscore (_).
2. Can contain letters, numbers, or underscores after the first character.
3. Cannot use reserved Python keywords (e.g., for, class, if).

```
# Invalid Variable Examples:
2name = "Bob" # SyntaxError: Starts with a number
class = 10    # SyntaxError: 'class' is a reserved keyword
```

Key Notes on Python Behavior:

- **Dynamically Typed:** Variable types do not need explicit declaration.
- **Case-Sensitive:** age and Age are two distinct variables.
- **Indentation:** Crucial for structure (standard is 4 spaces per indent level).
- **Keywords:** Reserved words with fixed meanings (e.g., if, for, class, while, else, in, def).

3. Data Types & Type Conversion

A data type defines the classification of a value stored in a variable.

Data Type	Keyword	Description	Example
Integer	int	Whole numbers (positive, negative, zero)	x = 10
Floating-point	float	Decimal / fractional numbers	PI = 3.14
String	str	Sequence of text characters	name = "shradha"
Boolean	bool	Truth values: True or False	isTeacher = True
NoneType	None	Special constant representing absence of value	empty_var = None

Use the built-in `type()` function to inspect variable types.

Implicit Conversion vs Explicit Type Casting

- **Type Conversion (Implicit):** Automatically performed by Python.

```
a = 5 # int
b = 3.0 # float
print(a + b) # Output: 8.0 (automatically converted to float)
```

- **Type Casting (Explicit):** Manually converted using functions like `int()`, `float()`, `str()`, `bool()`, `list()`, `tuple()`.

```
x = 10
y = float(x) # Converts int 10 to float 10.0
z = str(x)   # Converts int 10 to string "10"
```

4. Operators

An operator is a symbol directing Python to execute a specific mathematical or logical calculation on operands (e.g., in `1 + 3`, `+` is the operator, 1 and 3 are operands).

A. Arithmetic Operators

Used for standard mathematical operations: `+`, `-`, `*`, `/`, `%` (modulo/remainder), `**` (exponentiation).

B. Relational / Comparison Operators

Used to evaluate relationships: `==`, `!=`, `>`, `<`, `>=`, `<=`.

C. Assignment Operators

Used to store or update values: `=`, `+=`, `-=`, `*=`, `/=`, `%=`, `**=`.

D. Logical Operators

Used to combine boolean conditions: `and`, `or`, `not`.

val1	val2	val1 and val2	val1 or val2	not val1
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

E. Operator Precedence

1. `()` - Parentheses (Highest Priority)
2. `**` - Exponentiation
3. `+x`, `-x`, `~x` - Unary Operators
4. `*`, `/`, `//`, `%` - Multiplication, Division, Floor Division, Modulo
5. `+`, `-` - Addition, Subtraction
6. `<<`, `>>` - Bitwise Shifts
7. `&`, `^`, `|` - Bitwise AND, XOR, OR
8. `<`, `<=`, `>`, `>=`, `!=`, `==` - Comparison Operators
9. `not`, `and`, `or` - Logical Operators

10. =, +=, -=, etc. - Assignment Operators (Lowest Priority)

5. User Input & Practice Problem

The `input()` function captures input from the user while the program executes. Note that `input()` **always returns data as a string**, so explicit type casting is required for numerical calculations.

```
name = input("enter name: ")
print(name)
print(type(name)) # Output:
```

Practice Problem: Calculate Average of Two Numbers

Write a program that takes two numerical inputs (a & b) from the user and prints their average.

```
# Program to calculate average of two numbers
a = int(input("enter 1st num: ")) # Example input: 5
b = int(input("enter 2nd num: ")) # Example input: 10

avg = (a + b) / 2
print("average =", avg)           # Output: average = 7.5
```

✨ Keep Learning & Keep Exploring! ✨

Document created for GitHub profile: [gmlincoln](https://github.com/gmlincoln) (Golam Maula Lincoln)